Module 10 Assignment: Homework

Software Radio for Wireless Communications

1 Instructions

This assignment contains a design challenge requiring you to solve Error Coding problems.

What to turn in:
Turn in designs that meet the requirements specified below. **Turn in all scripts**.
Turn in a short write-up in which you explain your designs, show your results, and give instructions on how to run your scripts.

2 Assignment

2.1 Design Challenge: ITU-T G.9959 Checksum PER Measurement

30 points

In this design challenge you will implement a parity check. You will test that parity check, and then you will simulate Packet Rate Error as a function of SNR using that parity to detect errors.

ITU-T G.9959, the standard on which Z-Wave is based, uses an Odd Longitudinal Checksum for Error Detection.
The idea is illustrated in Figure 1. The code from the standard is shown in Figure 2.

The message is arranged into bytes. 8 parity bits are calculated, 1 parity bit for every bit position (column). All bits of a specific bit position across all bytes are XOR'd together. Then that total is XOR'd with 1. The example in Figure 1 has 4 bytes. All bits in bit position 0 (last column) are XOR'd together. That total equals 1. Then that total is XOR'd with 1 to yield 0.

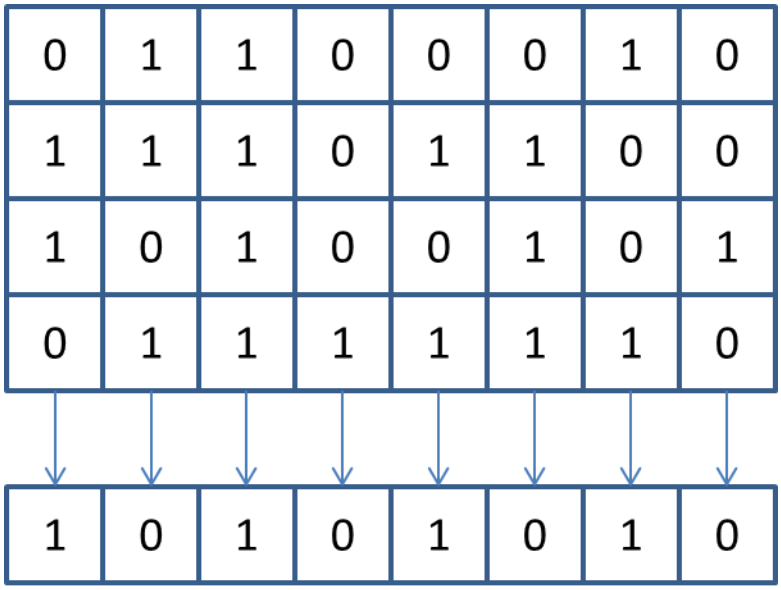

Figure 1 Odd Longitudinal Checksum

```
BYTE GenerateCheckSum(BYTE *Data, BYTE Length)
{
    BYTE CheckSum = 0xFF;
    for (; Length > 0; Length--)
        {
            CheckSum ^= *Data++;
        }
    return CheckSum;
}
```

Figure 2 Code from the Standard

This parity check appends 8 bits (1 byte) of parity bits to the end of the message bytes.

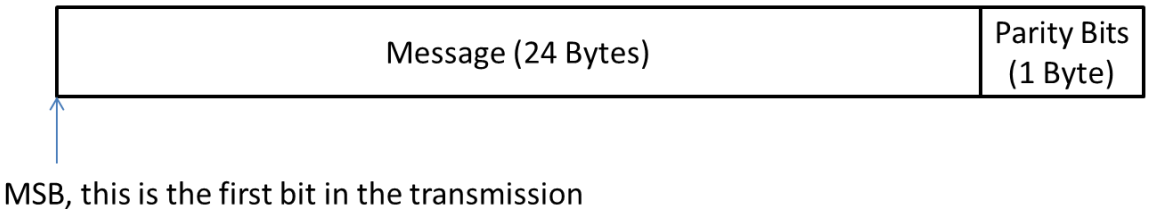

Figure 3 Message Format

76543210	76543210	76543210	76543210	76543210	●●●

Figure 4 Bit Positions

In this scheme, any bit error is a packet error. This scheme allows the receiver to detect using random messages, calculate the detected Packet Error Rate (PER) from SNR 5 db to 10 dB. Plot the true Packet Error Rate (PER) from SNR 5 db to 10 dB (any packet with any number of bit errors, whether the algorithm detects it or not) from SNR 5 db to 10 dB.

A *Packet* contains one message with parity bits appended (Payload+Parity).

Test your parity check as follows:

1) Make sure it is reversible: Create a random 192 bits long message, calculate the parity, then append the parity. Your packet is now 200 bits long. Calculate the parity again. The resulting second check on the 200 bits long packet should be zero.
2) Detect an error: Create a random 192 bits long message, calculate the parity, then append the parity. Your packet is now 200 bits long. Flip any one bit. Calculate the parity again. The resulting second check on the 200 bits long packet should be non-zero.

Simulation: Create a PER simulation like the 24 byte message CRC-16 example used in the Live Lab except that you will not use a CRC but you will use this parity check scheme to detect errors. If any error is detected in the packet, the whole packet counts as one packet error. The source for the CRC simulation is in the Live lab slides.

1) Loop over SNR values 0 to 6 dB in 1 dB steps using 10,000 packets per SNR value.
2) For each loop iteration, create a 192 bit long message (24 bytes)
3) Calculate the parity for that message and append it thus creating a packet
4) Apply modulation, noise, and demodulation
5) Calculate the parity on the packet that was received. If an error is detected, the whole packet is in error.
6) Remember to keep track of the *truth data*.

Show the difference between the truth data (actual number of packets with bit errors) and what your function detected.

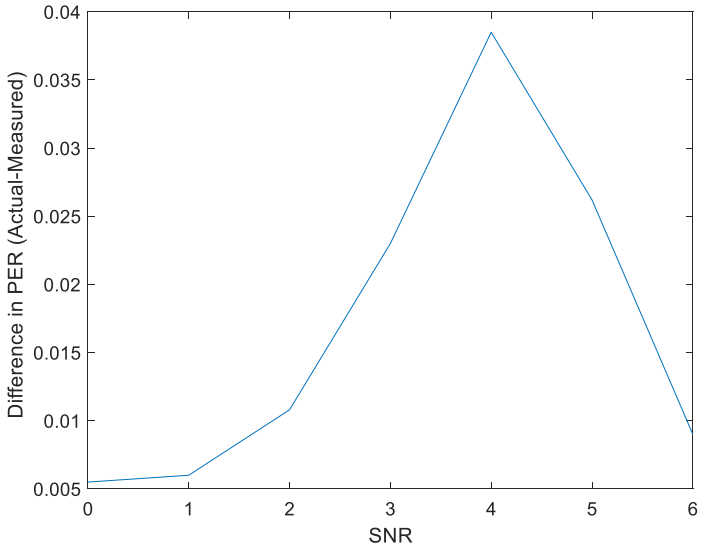

Figure 5 Difference in PER

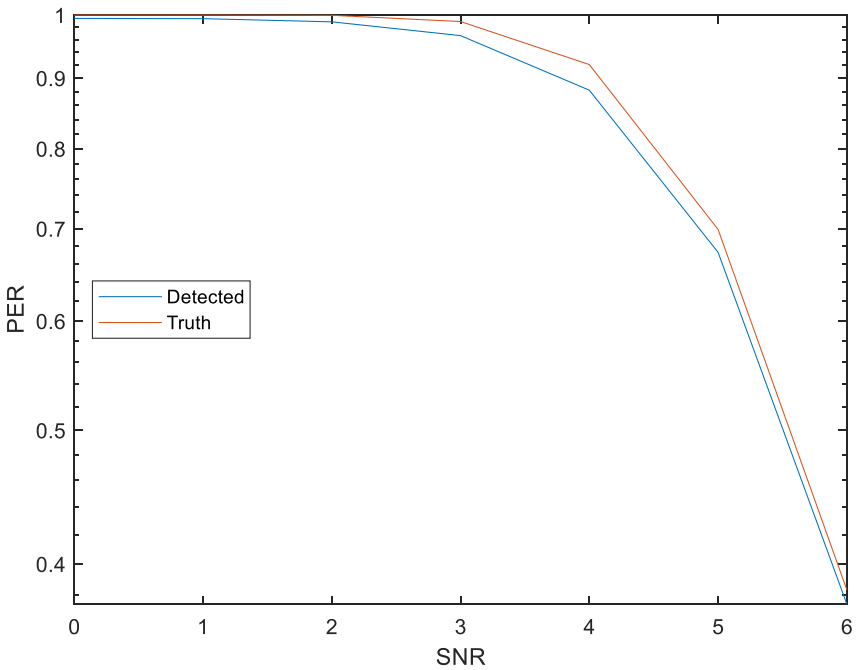

Figure 6 PER for the Parity Check

2.2 Experiment: A fair Hamming(7,4) Comparison

20 points

In the Live Lab we saw Hamming(7,4) offer a significant BER improvement, but at a cost of 7/4 times as much energy overall per packet.

Modify the Hamming(7,4) Simulation to adjust the energy per bit by 4/7ths. Show your results.

Compare your results against the theoretical BER curve for BPSK as developed in the Live Lab and the original Hamming(7,4) results (source in the slides).

What changed, and why?

2.3 Experiment: Soft Decoding

50 points

In this experiment you will compare hard and soft decoding. Hard decoding was done in the Live Lab and uses “Hard Decisions”. Soft Decoding will use the samples of the waveform before symbol decisions are made.

Step 1)
Hamming(7,4) converts 4-bit data vectors into 7-bit code words. There are only 16 valid 7-bit length code words.
Create a table of those 16 valid code words. Represent zeros as -1s.
Each of those codewords corresponds to a sequence of seven bits.

Step 2)
Break the BPSK signal into 7-symbol segments. Assume the BPSK signal is synchronized therefore use only the real component. Correlate those seven BPSK symbols against the table of valid codewords. The codeword yielding the maximum correlation is the one chosen for that 7-bit. Do not take the absolute value of the correlations.

Step 3)
Using the codeword that yields the maximum correlation for each length-7 vector of BPSK symbols, compare those bits to the known truth bits for that length-7 vector of BPSK symbols and record the frequency of bit errors.

The frequency of bit errors using soft decoding should be lower than that of hard decoding as a function of Eb/N0.